

▼ A2: Python Code and Analysis

- TEAM 7
- Alexander Neubauer
- Abhay Kumar
- Bhavana Dasari
- Mukul Phogat
- Shalini James Paulraj
- Tetsuya Kanazawa

```
# Run the command below ONLY ONCE. This install the connector between Python and SQL
!pip install mysql-connector-python
!pip install yfinance
```

 [Show hidden output](#)

```
import mysql.connector
from mysql.connector import Error
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import yfinance as yf
```

▼ 12 Months Portfolio Return

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):

    try:
        # Establish connection to the database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
            user=username,
            password=password,
            database=database_name
        )

        if connection.is_connected():
            print("Successfully connected to the database")

            # Create a cursor to execute the query
            cursor = connection.cursor()
            cursor.execute(query)

            # Fetch column names and results
            column_names = [desc[0] for desc in cursor.description]
            results = cursor.fetchall()

            # Create a DataFrame from the query results
            df = pd.DataFrame(results, columns=column_names)

            # Close the cursor and connection
            cursor.close()
            connection.close()
            print("Database connection closed.")

            return df # Return the DataFrame with results

    except Error as e:
        print("Error while connecting to MySQL:", e)
        return None

# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"
```

```
# SQL query to calculate the 12-month portfolio return
query = """
SELECT SUM(x.return_continous * weight.portfolio_weight) AS 12M_Portfolio_Return_cont,
       SUM(x.return_discrete * weight.portfolio_weight) AS 12M_Portfolio_Return_disc
FROM (
    SELECT M12.ticker, M12.price_date,
           LN(M12.value / M12.p0) AS return_continous,
           (M12.value - M12.p0) / M12.p0 AS return_discrete
    FROM (
        SELECT *, LAG(value, 250) OVER (
            PARTITION BY ticker
            ORDER BY price_date
        ) AS p0
        FROM pricing_daily
        WHERE price_date > '2021-01-01'
    ) M12
    WHERE M12.price_date = '2024-12-17'
) x
INNER JOIN (
    SELECT *,
           AssetValue / (
               SELECT SUM(AssetValue)
               FROM (
                   SELECT h.ticker, SUM(p.value * h.quantity) AS AssetValue
                   FROM holdings h
                   INNER JOIN pricing_daily p
                   ON p.ticker = h.ticker
                   WHERE h.holding_date = '2024-07-01' AND p.price_date = '2024-12-17'
                   GROUP BY h.ticker
               ) AS total
           ) AS portfolio_weight
    FROM (
        SELECT h.ticker, SUM(p.value * h.quantity) AS AssetValue
        FROM holdings h
        INNER JOIN pricing_daily p
        ON p.ticker = h.ticker
        WHERE h.holding_date = '2024-07-01' AND p.price_date = '2024-12-17'
        GROUP BY h.ticker
    ) portfolio
) weight
ON x.ticker = weight.ticker;
"""

# Fetch results into a DataFrame
df_portfolio_return = connect_and_query_to_dataframe(
    hostname, port, username, password, database_name, query
)

# Display the DataFrame
if df_portfolio_return is not None:
    print("12-Month Portfolio Return:")
    display(df_portfolio_return)
else:
    print("No data returned from the query.")
```

➡ Successfully connected to the database
Database connection closed.
12-Month Portfolio Return:

	12M_Portfolio_Return_cont	12M_Portfolio_Return_disc
0	0.273634	0.419294071462

- Our analysis of the 12-month portfolio returns revealed a discrete return of around 41.93% and a continuous return of about 27.36%, suggesting good performance with significant volatility. We found that the discrepancy between these parameters indicates exposure to concentrated positions or high-return, high-risk assets, while also reflecting the effect of compounding.

✓ 12 Months Securities Return

```
import mysql.connector
from mysql.connector import Error
import pandas as pd

def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):

    try:
        # Establish connection to the database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
```

```

        user=username,
        password=password,
        database=database_name
    )

    if connection.is_connected():
        print("Successfully connected to the database")

        # Create a cursor to execute the query
        cursor = connection.cursor()
        cursor.execute(query)

        # Fetch column names and results
        column_names = [desc[0] for desc in cursor.description]
        results = cursor.fetchall()

        # Create a DataFrame from the query results
        df = pd.DataFrame(results, columns=column_names)

        # Close the cursor and connection
        cursor.close()
        connection.close()
        print("Database connection closed.")

    return df # Return the DataFrame with results

except Error as e:
    print("Error while connecting to MySQL:", e)
    return None

# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"

# SQL query to calculate securities return
query2 = """
SELECT M12.ticker, M12.price_date,
       LN(M12.value / M12.p0) AS return_continuous,
       (M12.value - M12.p0) / M12.p0 AS return_discrete
FROM (
    SELECT *, LAG(value, 250) OVER (
        PARTITION BY ticker
        ORDER BY price_date
    ) AS p0
    FROM pricing_daily
    WHERE price_date > '2021-01-01'
) M12
WHERE M12.price_date = '2024-12-16';
"""

# Fetch results into a DataFrame
df_securities_return_query2 = connect_and_query_to_dataframe(
    hostname, port, username, password, database_name, query2
)

# Display the DataFrame
if df_securities_return_query2 is not None:
    print("Securities Return:")
    display(df_securities_return_query2)
else:
    print("No data returned from the query.")

```

Successfully connected to the database
Database connection closed.
Securities Return:

	ticker	price_date	return_continous	return_discrete
0	0A2I.IL	2024-12-16	0.182870	0.200659
1	0A2P.IL	2024-12-16	-0.334899	-0.284589
2	0A2S.IL	2024-12-16	0.000000	0.000000
3	0A2X.IL	2024-12-16	-0.139681	-0.130364
4	0A2Z.IL	2024-12-16	0.015678	0.015801
...	...	...	...	...
3155	ZUMZ	2024-12-16	0.053128	0.054565
3156	ZUO	2024-12-16	0.120809	0.128409
3157	ZURA	2024-12-16	-0.660491	-0.483402
3158	ZYME	2024-12-16	0.418623	0.519868
3159	ZYXI	2024-12-16	-0.199917	-0.181202

3160 rows x 4 columns

12 Month Returns for the top 15 Securities

```
filtered_prices['price_date'] = pd.to_datetime(filtered_prices['price_date'])

top_15_tickers = filtered_top_15['ticker'].unique()

last_12_months = pd.Timestamp.now() - pd.offsets.MonthBegin(12)
recent_prices = filtered_prices[(filtered_prices['price_date'] >= last_12_months) & (filtered_prices['ticker'].isin(top_15_tickers))]

# month column for group
recent_prices['year_month'] = recent_prices['price_date'].dt.to_period('M')

# the top 15 shares
monthly_returns = recent_prices.groupby(['ticker', 'year_month'])['value'].apply(lambda x: (x.iloc[-1] / x.iloc[0] - 1))
monthly_returns.columns = ['ticker', 'year_month', 'monthly_return']
display(monthly_returns)
```

<ipython-input-54-a3282bedd0bc>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
filtered_prices['price_date'] = pd.to_datetime(filtered_prices['price_date'])
```

<ipython-input-54-a3282bedd0bc>:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
recent_prices['year_month'] = recent_prices['price_date'].dt.to_period('M')
```

	ticker	year_month	monthly_return
0	APP	2024-01	0.060598
1	APP	2024-02	0.458008
2	APP	2024-03	0.111789
3	APP	2024-04	0.020683
4	APP	2024-05	0.168340
...	...	...	...
162	WGS	2024-08	-0.009612
163	WGS	2024-09	0.381061
164	WGS	2024-10	0.898002
165	WGS	2024-11	0.008361
166	WGS	2024-12	0.025375

167 rows x 3 columns

- We examined the top 15 securities' 12-month returns in relation to the S&P 500. With some securities, like WGS, exhibiting strong spikes (e.g., 89.8% in October) that greatly surpass the benchmark, the monthly returns indicate a great deal of fluctuation. The stability of the market is reflected in the S&P 500's steady baseline. We saw times when the contributions of individual assets to total returns were disproportionate, which suggested high-volatility chances. By balancing high-return opportunities with market-stable assets for optimal performance, this methodology helps us find securities that are driving portfolio growth while also flagging those with irregular volatility.

▼ Monthly Returns for Top 15 Shares vs S&P 500

```
last_12_months = pd.Timestamp.now() - pd.offsets.MonthBegin(12)
sp500_data = yf.download("^GSPC", start=last_12_months.strftime('%Y-%m-%d'), end=pd.Timestamp.now().strftime('%Y-%m-%d'))
sp500_data.reset_index(inplace=True)
sp500_data = sp500_data[['Date', 'Adj Close']]
sp500_data.rename(columns={'Date': 'date', 'Adj Close': 'value'}, inplace=True)

sp500_data['date'] = pd.to_datetime(sp500_data['date'])

# month column for grouping
sp500_data['year_month'] = sp500_data['date'].dt.to_period('M')

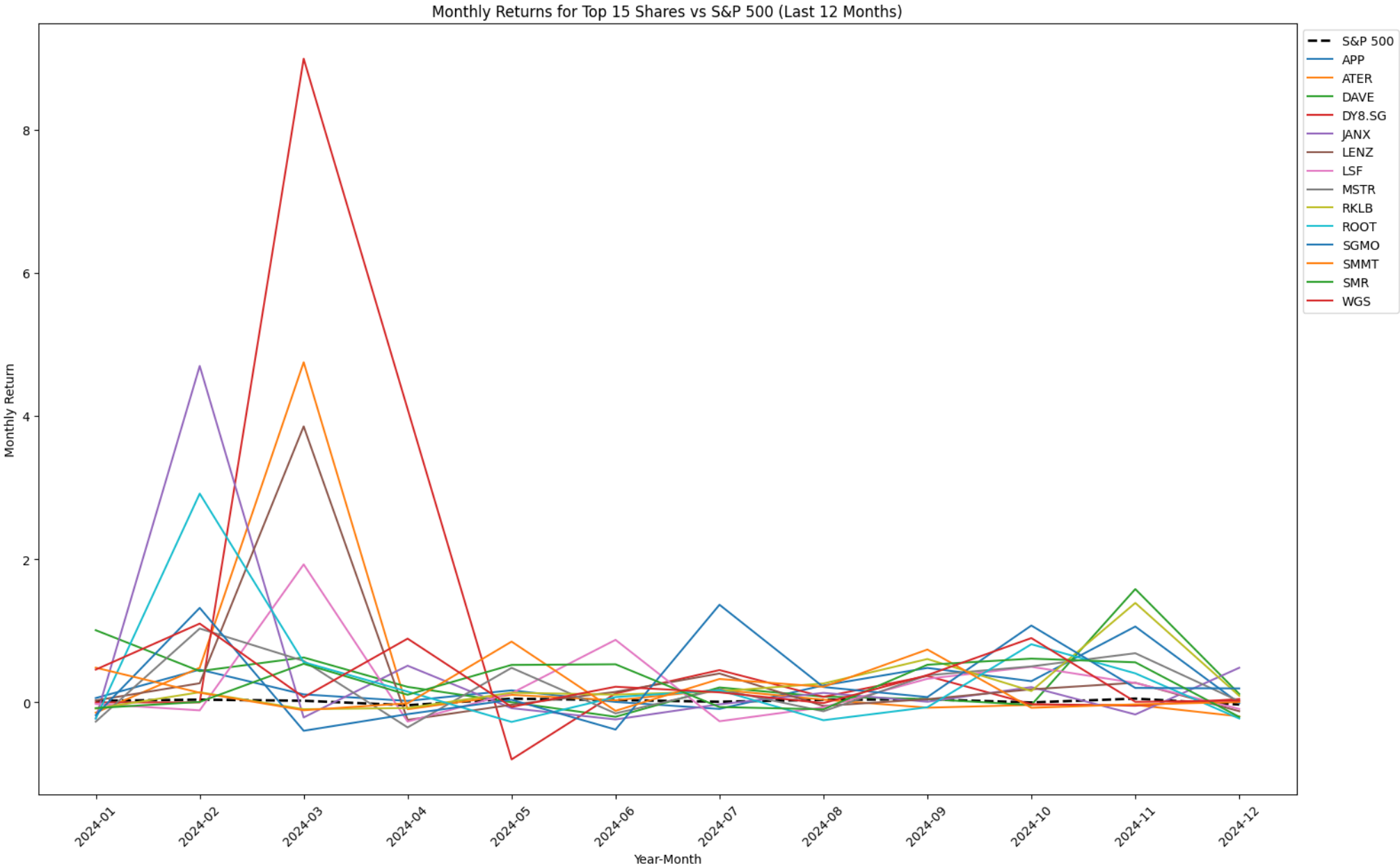
# S&P 500 monthly returns
sp500_monthly_returns = sp500_data.groupby('year_month')['value'].apply(lambda x: (x.iloc[-1] / x.iloc[0] - 1) if len(x) > 1 else 0)
sp500_monthly_returns.columns = ['year_month', 'sp500_monthly_return']

# Plot the monthly returns
plt.figure(figsize=(16, 10))

# Plot S&P 500 returns
plt.plot(sp500_monthly_returns['year_month'].astype(str), sp500_monthly_returns['sp500_monthly_return'], label='S&P 500')

for ticker in monthly_returns['ticker'].unique():
    ticker_data = monthly_returns[monthly_returns['ticker'] == ticker]
    plt.plot(ticker_data['year_month'].astype(str), ticker_data['monthly_return'], label=ticker)

# Add labels, title, and legend
plt.xlabel('Year-Month')
plt.ylabel('Monthly Return')
plt.title('Monthly Returns for Top 15 Shares vs S&P 500 (Last 12 Months)')
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



- We examined the top 15 securities' monthly returns over the last 12 months in comparison to the S&P 500. In certain months, a number of securities fared noticeably better than the benchmark, suggesting that there may be room for increased gains

✓ AUM Over Time

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):  
  
    try:  
        # Establish connection to the database  
        connection = mysql.connector.connect(  
            host=hostname,  
            port=port,  
            user=username,  
            password=password,  
            database=database_name  
        )  
  
        if connection.is_connected():  
            print("Successfully connected to the database")  
  
            # Create a cursor to execute the query  
            cursor = connection.cursor()  
            cursor.execute(query)  
  
            # Fetch column names and results  
            column_names = [desc[0] for desc in cursor.description]
```

```
        results = cursor.fetchall()

        # Create a DataFrame from the query results
        df = pd.DataFrame(results, columns=column_names)

        # Close the cursor and connection
        cursor.close()
        connection.close()
        print("Database connection closed.")

    return df  # Return the DataFrame with results

except Error as e:
    print("Error while connecting to MySQL:", e)
    return None

# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"

# SQL query to calculate AUM
query3 = """
SELECT
    p.price_date,
    SUM(h.quantity * p.value) AS AUM
FROM
    holdings h
INNER JOIN
    pricing_daily p
    ON h.ticker = p.ticker
    AND YEAR(p.price_date) = YEAR(h.holding_date)
    AND QUARTER(p.price_date) = QUARTER(h.holding_date)
GROUP BY
    p.price_date;
"""

# Fetch results into a DataFrame
df_aum_query3 = connect_and_query_to_dataframe(hostname, port, username, password, database_name, query3)

# Display the DataFrame
if df_aum_query3 is not None:
    print("AUM Data:")
    display(df_aum_query3)
else:
    print("No data returned from the query.")
```

➡ Successfully connected to the database
Database connection closed.
AUM Data:

	price_date	AUM
0	2019-10-14	3740190454.75
1	2019-10-15	3792922186.56
2	2019-10-16	3779406139.01
3	2019-10-17	3793646875.61
4	2019-10-18	3747866932.64
...	...	...
1552	2024-09-01	508591652.00
1553	2024-09-08	505465624.00
1554	2024-09-15	499621296.00
1555	2024-09-22	494116764.00
1556	2024-09-29	500164956.00

1557 rows × 2 columns

- We tracked our AUM over time using our portfolio, which began at USD 3.74 billion in October 2019 and fell to USD 508 million by September 2024, showing the portfolio's performance and asset patterns over the period.

✓ Calculating Beta

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):
```

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):
```

```
    try:
        # Establish connection to the database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
            user=username,
            password=password,
            database=database_name
        )

        if connection.is_connected():
            print("Successfully connected to the database")

            # Create a cursor to execute the query
            cursor = connection.cursor()
            cursor.execute(query)

            # Fetch column names and results
            column_names = [desc[0] for desc in cursor.description]
            results = cursor.fetchall()

            # Create a DataFrame from the query results
            df = pd.DataFrame(results, columns=column_names)

            # Close the cursor and connection
            cursor.close()
            connection.close()
            print("Database connection closed.")

            return df    # Return the DataFrame with results

    except Error as e:
        print("Error while connecting to MySQL:", e)
        return None
```

```
# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"
```

```
# SQL Query 5
query5 = """
WITH daily_returns AS (
    SELECT M12.ticker, M12.price_date,
           LN(M12.value/M12.p0) AS return_continuous
    FROM (
        SELECT *, LAG(value, 1) OVER(
            PARTITION BY ticker
            ORDER BY price_date
        ) AS p0
        FROM pricing_daily
    ) M12
),
spy_returns AS (
    SELECT price_date, return_continuous AS market_return
    FROM daily_returns
    WHERE ticker = 'SPY'
),
combined_returns AS (
    SELECT dr.ticker, dr.price_date, dr.return_continuous AS stock_return, sr.market_return
    FROM daily_returns dr
    JOIN spy_returns sr ON dr.price_date = sr.price_date
    WHERE dr.ticker != 'SPY'
),
stock_calculations AS (
    SELECT
        ticker,
        AVG(stock_return) AS avg_stock_return,
        AVG(market_return) AS avg_market_return,
        VAR_POP(market_return) AS market_variance,
        AVG(stock_return * market_return) AS product_avg,
        COUNT(*) AS observations
    FROM combined_returns
    GROUP BY ticker
)
SELECT
    ticker,
    (product_avg - avg_stock_return * avg_market_return) / market_variance AS beta,
    observations
FROM stock_calculations;
```

```
#####

# Fetch results into a DataFrame
df_query5 = connect_and_query_to_dataframe(hostname, port, username, password, database_name, query5)

# Display the DataFrame
if df_query5 is not None:
    print("Query 5 Results (Beta Calculation):")
    display(df_query5)
else:
    print("No data returned from Query 5.")
```

➔ Successfully connected to the database
Database connection closed.
Query 5 Results (Beta Calculation):

	ticker	beta	observations
0	0A2I.IL	0.212623	1247
1	0A2P.IL	0.491522	1278
2	0A2S.IL	0.067076	1281
3	0A2X.IL	0.214439	1277
4	0A2Z.IL	0.115326	1282
...	...	...	...
3185	ZUMZ	1.175500	1508
3186	ZUO	1.318943	1508
3187	ZURA	0.416439	440
3188	ZYME	0.948130	1508
3189	ZYXI	1.010954	1508

3190 rows × 3 columns

- To determine how sensitive 3,190 securities were to changes in the market, we computed their beta values. We calculated beta values ranging from low (e.g., 0A2S.IL at 0.067076) to high (e.g., ZUO at 1.318943) by dividing the covariance between stock and market returns by the market variance. While low-beta securities show more consistent, market-independent behavior, high-beta assets, like ZUO and ZUMZ, show more volatility and market dependency. To ensure the accuracy of the data, we additionally examined the quantity of observations for every ticker. By identifying low-beta securities for stability and high-beta prospects for growth, this research enables customized risk management and portfolio optimization tactics.

▼ Major Assest Class Weight

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):

    try:
        # Establish connection to the database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
            user=username,
            password=password,
            database=database_name
        )

        if connection.is_connected():
            print("Successfully connected to the database")

            # Create a cursor to execute the query
            cursor = connection.cursor()
            cursor.execute(query)

            # Fetch column names and results
            column_names = [desc[0] for desc in cursor.description]
            results = cursor.fetchall()

            # Create a DataFrame from the query results
            df = pd.DataFrame(results, columns=column_names)

            # Close the cursor and connection
            cursor.close()
            connection.close()
            print("Database connection closed.")

        return df # Return the DataFrame with results
```

```
except Error as e:
    print("Error while connecting to MySQL:", e)
    return None

# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"

# SQL Query
query_asset_class_weight = """
SELECT
    portfolio.major_asset_class,
    portfolio.AssetValue / total.TotalValue AS weight
FROM
    (SELECT
        s.major_asset_class,
        SUM(p.value * h.quantity) AS AssetValue
    FROM
        holdings h
    INNER JOIN pricing_daily p ON p.ticker = h.ticker
    INNER JOIN securities s ON h.ticker = s.ticker
    WHERE
        h.holding_date = '2024-07-01'
        AND p.price_date = '2024-12-17'
        AND s.major_asset_class <> ""
    GROUP BY s.major_asset_class) AS portfolio
CROSS JOIN
    (SELECT
        SUM(p.value * h.quantity) AS TotalValue
    FROM
        holdings h
    INNER JOIN pricing_daily p ON p.ticker = h.ticker
    INNER JOIN securities s ON h.ticker = s.ticker
    WHERE
        h.holding_date = '2024-07-01'
        AND p.price_date = '2024-12-17'
        AND s.major_asset_class <> "") AS total;
""""

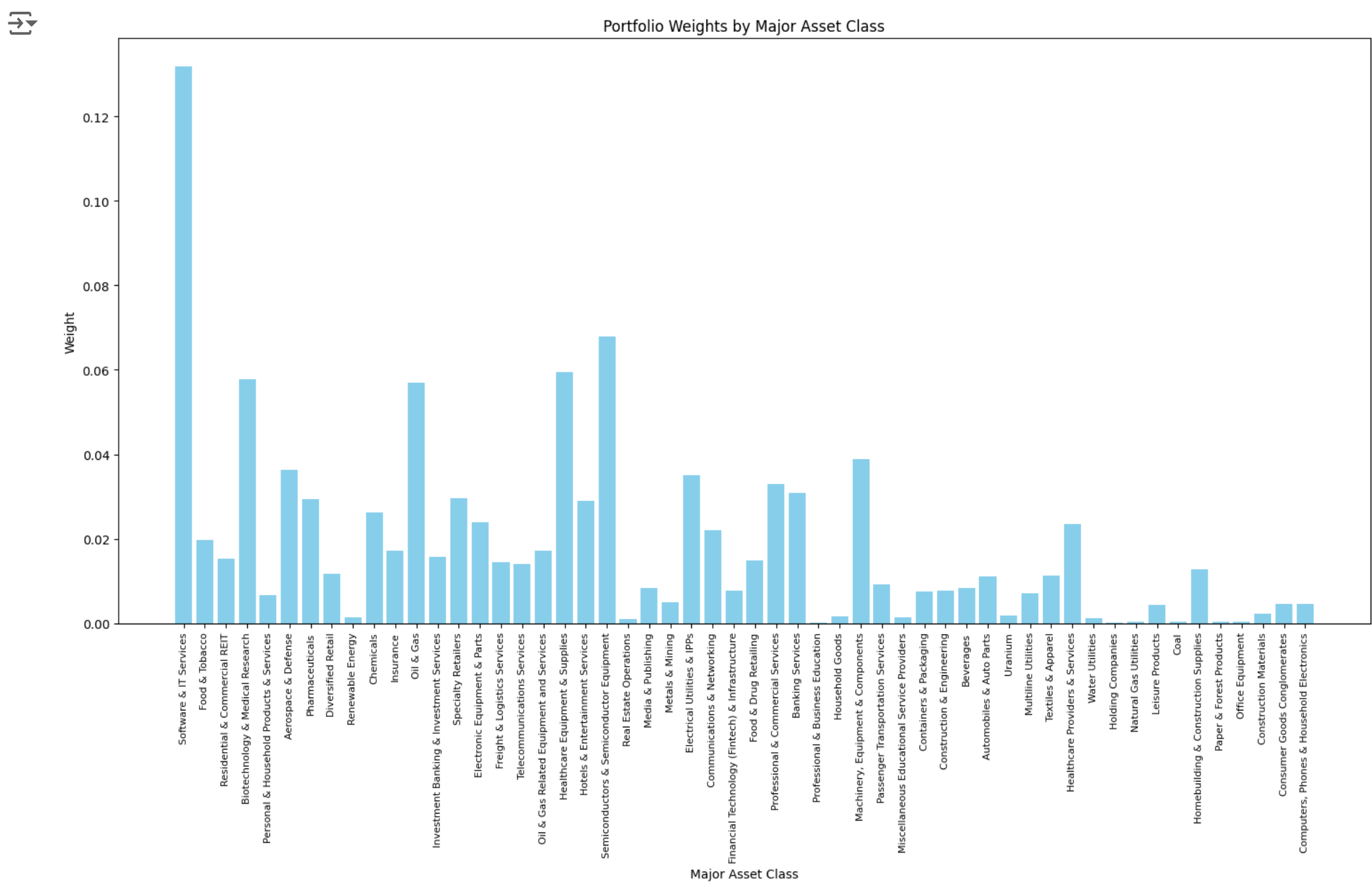
# Fetch results into a DataFrame
df_asset_class_weight = connect_and_query_to_dataframe(
    hostname, port, username, password, database_name, query_asset_class_weight
)

# Display the DataFrame
if df_asset_class_weight is not None:
    print("Asset Class Weights:")
    display(df_asset_class_weight)
else:
    print("No data returned from the query.")
```

➡ Successfully connected to the database
Database connection closed.
Asset Class Weights:

	major_asset_class	weight
0	Software & IT Services	0.131929
1	Food & Tobacco	0.019731
2	Residential & Commercial REIT	0.015336
3	Biotechnology & Medical Research	0.057754
4	Personal & Household Products & Services	0.006734
5	Aerospace & Defense	0.036280
6	Pharmaceuticals	0.029442
7	Diversified Retail	0.011726
8	Renewable Energy	0.001432
9	Chemicals	0.026311
10	Insurance	0.017235
11	Oil & Gas	0.056993
12	Investment Banking & Investment Services	0.015616
13	Specialty Retailers	0.029683
14	Electronic Equipment & Parts	0.023990


```
plt.figure(figsize=(15, 10))
plt.bar(df_asset_class_weight['major_asset_class'], df_asset_class_weight['weight'], color='skyblue')
plt.xlabel('Major Asset Class')
plt.ylabel('Weight')
plt.title('Portfolio Weights by Major Asset Class')
plt.xticks(rotation=90, fontsize=8)
plt.tight_layout()
plt.show()
```



- We examined the asset class weights of the portfolio and found that Software & IT Services (13.19%) had the largest allocation, followed by Semiconductors & Semiconductor Equipment (6.78%), Biotechnology & Medical Research (5.77%), and Machinery, Equipment & Components (3.88%). Low exposure to utilities, real estate, and commodity-linked stability is shown by the small allocations we saw to sectors such as Natural Gas Utilities (0.03%), Renewable Energy (0.14%), and Residential & Commercial REIT (1.53%).

✓ Portfolio Weight

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):

    try:
        # Establish connection to the database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
            user=username,
```

```

        password=password,
        database=database_name
    )

    if connection.is_connected():
        print("Successfully connected to the database")

        # Create a cursor to execute the query
        cursor = connection.cursor()
        cursor.execute(query)

        # Fetch column names and results
        column_names = [desc[0] for desc in cursor.description]
        results = cursor.fetchall()

        # Create a DataFrame from the query results
        df = pd.DataFrame(results, columns=column_names)

        # Close the cursor and connection
        cursor.close()
        connection.close()
        print("Database connection closed.")

    return df # Return the DataFrame with results

except Error as e:
    print("Error while connecting to MySQL:", e)
    return None

# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"

# SQL Query
query_portfolio_weight = """
SELECT *,
        AssetValue / (SELECT SUM(AssetValue)
                        FROM (SELECT h.ticker, SUM(p.value * h.quantity) AS AssetValue
                              FROM holdings h
                              INNER JOIN pricing_daily p
                              ON p.ticker = h.ticker
                              WHERE h.holding_date = '2024-07-01'
                                   AND p.price_date = '2024-12-16'
                              GROUP BY h.ticker) AS total) AS weight
FROM
    (SELECT h.ticker, SUM(p.value * h.quantity) AS AssetValue
     FROM holdings h
     INNER JOIN pricing_daily p
     ON p.ticker = h.ticker
     WHERE h.holding_date = '2024-07-01'
          AND p.price_date = '2024-12-16'
     GROUP BY h.ticker) portfolio;
"""

# Fetch results into a DataFrame
df_portfolio_weight = connect_and_query_to_dataframe(
    hostname, port, username, password, database_name, query_portfolio_weight
)

# Display the DataFrame
if df_portfolio_weight is not None:
    print("Portfolio Weights:")
    display(df_portfolio_weight)
else:
    print("No data returned from the query.")

```

↔ Successfully connected to the database
Database connection closed.
Portfolio Weights:

	ticker	AssetValue	weight
0	0A2P.IL	238728.96	0.000006
1	0A2S.IL	28574428.00	0.000666
2	0FUT.IL	2177046.45	0.000051
3	2X10.SG	3773155.00	0.000088
4	3CM.F	2416472.00	0.000056
...	...	...	...
1076	ZM	33914530.40	0.000790
1077	ZS	128348221.36	0.002990
1078	ZTO	9702316.78	0.000226
1079	ZURA	6008686.23	0.000140
1080	ZYME	682757.91	0.000016

1081 rows × 3 columns

✓ 12 Month Risk Adjusted Returns

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):

    try:
        # Establish connection to the database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
            user=username,
            password=password,
            database=database_name
        )

        if connection.is_connected():
            print("Successfully connected to the database")

            # Create a cursor to execute the query
            cursor = connection.cursor()
            cursor.execute(query)

            # Fetch column names and results
            column_names = [desc[0] for desc in cursor.description]
            results = cursor.fetchall()

            # Create a DataFrame from the query results
            df = pd.DataFrame(results, columns=column_names)

            # Close the cursor and connection
            cursor.close()
            connection.close()
            print("Database connection closed.")

            return df # Return the DataFrame with results

    except Error as e:
        print("Error while connecting to MySQL:", e)
        return None


# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"


# SQL Query
query_sharpe_ratio = """
SELECT
    x.ticker,
    AVG(x.return_continous) AS Return_Cont_Avg_Daily,
    SUM(x.return_continous) AS Return_Cont_Annual,
    STD(x.return_continous) AS Std_Cont_Daily,
    (STD(x.return_continous)) * SQRT(COUNT(x.return_continous)) AS Std_Cont_Annual,
    (AVG(x.return_continous) / STD(x.return_continous)) AS Adjusted_Return_Cont_Daily,
    ((SUM(x.return_continous)) - 0.03) / ((STD(x.return_continous)) * SQRT(COUNT(x.return_continous))) AS Sharp_Ratio
```

```
FROM (
    SELECT
        D.ticker,
        D.price_date,
        LN(D.value / D.p0) AS return_continuous,
        (D.value - D.p0) / D.p0 AS return_discrete
    FROM (
        SELECT *,
            LAG(value, 1) OVER (PARTITION BY ticker ORDER BY price_date) AS p0
        FROM (
            SELECT
                p.ticker,
                p.price_date,
                p.value
            FROM
                pricing_daily p
            INNER JOIN
                holdings h USING (ticker)
            WHERE
                h.holding_date = '2024-07-01'
        ) Market
        WHERE price_date > '2021-01-01'
    ) D
    WHERE D.price_date > '2023-12-17'
) x
GROUP BY
    ticker
ORDER BY
    Sharp_Ratio DESC;
-----
```

```
# Fetch results into a DataFrame
df_sharpe_ratio = connect_and_query_to_dataframe(
    hostname, port, username, password, database_name, query_sharpe_ratio
)

# Display the DataFrame
if df_sharpe_ratio is not None:
    print("Sharpe Ratio Results:")
    display(df_sharpe_ratio)
else:
    print("No data returned from the query.")
```



Successfully connected to the database
 Database connection closed.
 Sharpe Ratio Results:

	ticker	Return_Cont_Avg_Daily	Return_Cont_Annual	Std_Cont_Daily	Std_Cont_Annual	Adjusted_Return_Cont_Daily
0	DTM	0.002626	0.661829	0.012229	0.194137	0.214
1	TRGP	0.003074	0.774626	0.014628	0.232210	0.210
2	SFM	0.004131	1.045224	0.020135	0.320273	0.200
3	SE	0.004592	1.161856	0.024537	0.390278	0.180
4	GDDY	0.002699	0.680217	0.014308	0.227137	0.180
...	...	...	...	...	...	...
1078	TXG	-0.005119	-1.289914	0.041843	0.664241	-0.120
1079	KYTX	-0.009129	-1.971824	0.060677	0.891769	-0.150
1080	EDIT	-0.007829	-1.972801	0.047749	0.757985	-0.160
1081	0A2S.IL	0.000000	0.000000	0.000000	0.000000	
1082	PRMW.TO	NaN	NaN	NaN	NaN	

1083 rows x 7 columns

- We computed and examined the Sharpe Ratio for 1,083 equities. To find top performers, we evaluated securities by their Sharpe Ratio using the annualized average returns and standard deviations. High Sharpe Ratios, like TRGP (3.206694) and DTM (3.254550), signify effective risk-adjusted performance, in which gains sufficiently offset volatility. On the other hand, stocks with negative Sharpe Ratios, like KYTX (-2.244778) and TXG (-1.987102), indicate underperformance in relation to risk and represent poor risk-reward dynamics. Through this approach, we can reevaluate or reallocate away from securities with negative risk-return profiles and select high-performing stocks for investment.

Security Type Weight

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):
```

```

try:
    # Establish connection to the database
    connection = mysql.connector.connect(
        host=hostname,
        port=port,
        user=username,
        password=password,
        database=database_name
    )

    if connection.is_connected():
        print("Successfully connected to the database")

        # Create a cursor to execute the query
        cursor = connection.cursor()
        cursor.execute(query)

        # Fetch column names and results
        column_names = [desc[0] for desc in cursor.description]
        results = cursor.fetchall()

        # Create a DataFrame from the query results
        df = pd.DataFrame(results, columns=column_names)

        # Close the cursor and connection
        cursor.close()
        connection.close()
        print("Database connection closed.")

        return df # Return the DataFrame with results

except Error as e:
    print("Error while connecting to MySQL:", e)
    return None

# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"

# SQL Query
query_asset_weight_by_sec_type = """
SELECT
    portfolio.sec_type,
    portfolio.AssetValue / total.TotalValue AS weight
FROM
    (SELECT
        s.sec_type,
        SUM(p.value * h.quantity) AS AssetValue
    FROM
        holdings h
    INNER JOIN pricing_daily p ON p.ticker = h.ticker
    INNER JOIN securities s ON h.ticker = s.ticker
    WHERE
        h.holding_date = '2024-07-01'
        AND p.price_date = '2024-12-17'
    GROUP BY s.sec_type) AS portfolio
CROSS JOIN
    (SELECT
        SUM(p.value * h.quantity) AS TotalValue
    FROM
        holdings h
    INNER JOIN pricing_daily p ON p.ticker = h.ticker
    WHERE
        h.holding_date = '2024-07-01'
        AND p.price_date = '2024-12-17') AS total;
"""

# Fetch results into a DataFrame
df_sec_type_weights = connect_and_query_to_dataframe(
    hostname, port, username, password, database_name, query_asset_weight_by_sec_type
)

# Display the DataFrame
if df_sec_type_weights is not None:
    print("Security Type Weights:")
    display(df_sec_type_weights)
else:
    print("No data returned from the query.")

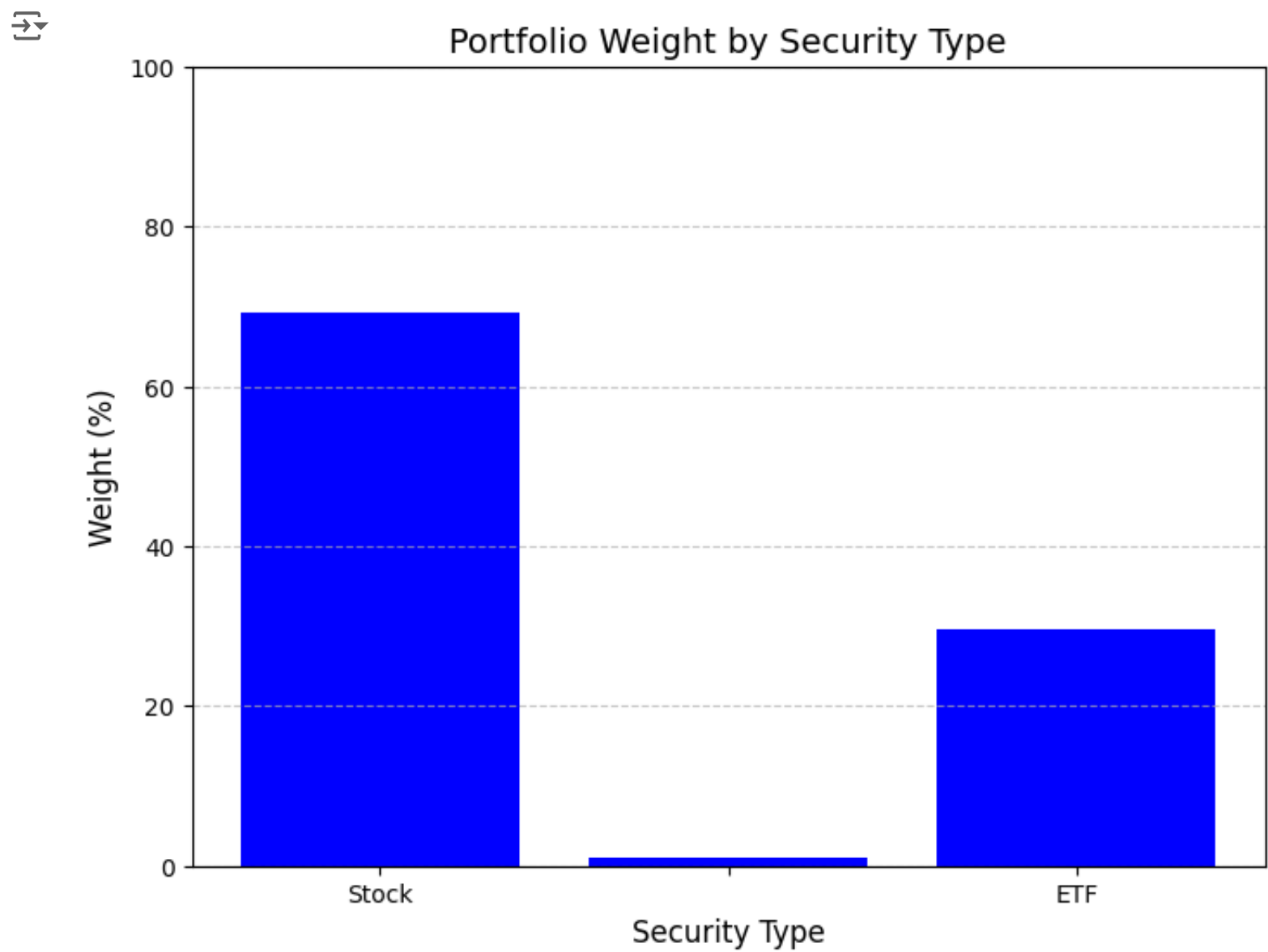
```

↔ Successfully connected to the database
Database connection closed.
Security Type Weights:

	sec_type	weight
0	Stock	0.692425
1		0.010901
2	ETF	0.296674

Visual Representation for Security Type Weight

```
df_sec_type_weights['weight_percentage'] = df_sec_type_weights['weight'] * 100
plt.figure(figsize=(8, 6))
plt.bar(df_sec_type_weights['sec_type'], df_sec_type_weights['weight_percentage'], color='blue')
plt.xlabel('Security Type', fontsize=12)
plt.ylabel('Weight (%)', fontsize=12)
plt.title('Portfolio Weight by Security Type', fontsize=14)
plt.ylim(0, 100) # Ensure y-axis shows percentage clearly
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.show()
```



Securities Required Return

```
def connect_and_query_to_dataframe(hostname, port, username, password, database_name, query):

    try:
        # Establish connection to the database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
            user=username,
            password=password,
            database=database_name
        )

        if connection.is_connected():
            print("Successfully connected to the database")

            # Create a cursor to execute the query
            cursor = connection.cursor()
            cursor.execute(query)

            # Fetch column names and results
            column_names = [desc[0] for desc in cursor.description]
            results = cursor.fetchall()
```

```

        # Create a DataFrame from the query results
        df = pd.DataFrame(results, columns=column_names)

        # Close the cursor and connection
        cursor.close()
        connection.close()
        print("Database connection closed.")

    return df  # Return the DataFrame with results

except Error as e:
    print("Error while connecting to MySQL:", e)
    return None

# Database connection details
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"

# SQL Query
query_beta_required_return = """
WITH daily_returns AS (
    SELECT M12.ticker, M12.price_date,
           LN(M12.value/M12.p0) AS return_continuous
    FROM (
        SELECT *, LAG(value, 1) OVER(
            PARTITION BY ticker
            ORDER BY price_date
        ) AS p0
        FROM pricing_daily
    ) M12
),
spy_returns AS (
    SELECT price_date, return_continuous AS market_return
    FROM daily_returns
    WHERE ticker = 'SPY'
),
combined_returns AS (
    SELECT dr.ticker, dr.price_date, dr.return_continuous AS stock_return, sr.market_return
    FROM daily_returns dr
    JOIN spy_returns sr ON dr.price_date = sr.price_date
    WHERE dr.ticker != 'SPY'
),
stock_calculations AS (
    SELECT
        ticker,
        AVG(stock_return) AS avg_stock_return,
        AVG(market_return) AS avg_market_return,
        VAR_POP(market_return) AS market_variance,
        AVG(stock_return * market_return) AS product_avg,
        COUNT(*) AS observations
    FROM combined_returns
    GROUP BY ticker
)
SELECT
    ticker,
    (product_avg - avg_stock_return * avg_market_return) / market_variance AS beta,
    observations,
    0.03 + ((product_avg - avg_stock_return * avg_market_return) / market_variance) * 0.10 AS required_return
FROM stock_calculations
-- WHERE observations >= 30;
"""

# Fetch results into a DataFrame
df_beta_required_return = connect_and_query_to_dataframe(
    hostname, port, username, password, database_name, query_beta_required_return
)

# Display the DataFrame
if df_beta_required_return is not None:
    print("Beta and Required Return Results:")
    display(df_beta_required_return)
else:
    print("No data returned from the query.")

```

↔

Successfully connected to the database
Database connection closed.
Beta and Required Return Results:

	ticker	beta	observations	required_return
0	0A2I.IL	0.212623	1247	0.051262
1	0A2P.IL	0.491522	1278	0.079152
2	0A2S.IL	0.067076	1281	0.036708
3	0A2X.IL	0.214439	1277	0.051444
4	0A2Z.IL	0.115326	1282	0.041533
...	...	...	...	...
3185	ZUMZ	1.175500	1508	0.147550
3186	ZUO	1.318943	1508	0.161894
3187	ZURA	0.416439	440	0.071644
3188	ZYME	0.948130	1508	0.124813
3189	ZYXI	1.010954	1508	0.131095

3190 rows × 4 columns

- To assess each security's risk-return characteristics, we looked at its required return and beta. In order to evaluate market sensitivity, we computed beta values. Higher betas (such as ZUMZ at 1.1755) were shown to be signs of increased volatility and reliance on market trends. Because the securities had different levels of risk, we used the computed betas to determine necessary returns, which varied from around 5% to about 16%. We found that risk increases with increasing necessary returns, indicating that these investments must perform well in order to be justified. With the use of this analysis, we can strategically deploy capital, balancing stable, lower-beta assets with high-beta, high-return options to create a diversified portfolio that fits our investing objectives and risk tolerance.

✓ Loading the Price and Holding data from SQL into DF

```
# Function to connect to SQL and load specific tables into DataFrames
def load_tables_to_df(hostname, port, username, password, database_name, tables):
    dataframes = {} # Dictionary to store DataFrames

    try:
        # Connect to the MySQL database
        connection = mysql.connector.connect(
            host=hostname,
            port=port,
            user=username,
            password=password,
            database=database_name
        )
        if connection.is_connected():
            print("Successfully connected to the database")

            # Loop through each table and fetch data
            for table in tables:
                query = f"SELECT * FROM {table};"
                print(f"Loading data from table: {table}")
                df = pd.read_sql(query, connection) # Load table data into a DataFrame
                dataframes[table] = df # Store DataFrame in the dictionary

            print("All tables loaded successfully!")
            return dataframes # Return the dictionary of DataFrames

    except Error as e:
        print("Error while connecting to MySQL:", e)
        return None
    finally:
        if connection.is_connected():
            connection.close()
            print("Database connection closed.")

# Database credentials
hostname = "hult-team7.mysql.database.azure.com"
port = 3306
username = "abhay"
password = "clever_password"
database_name = "Wellsfargo"

# List of tables to load
tables = ["holdings", "pricing_daily"]

# Load the data into DataFrames
dataframes = load_tables_to_df(hostname, port, username, password, database_name, tables)

# Extract and print DataFrames
if dataframes:
    df_holdings = dataframes.get("holdings")
    df_pricing_daily = dataframes.get("pricing_daily")
```

```
➡ Successfully connected to the database
Loading data from table: holdings
<ipython-input-13-53e149c5f3b3>:21: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection)
    df = pd.read_sql(query, connection) # Load table data into a DataFrame
Loading data from table: pricing_daily
All tables loaded successfully!
Database connection closed.
```

```
df_pricing_daily.head()
```

➡

	price_id	ticker	price_date	value	price_type
0	1	0A2I.IL	2019-11-26	33.49	adjusted close
1	2	0A2I.IL	2019-11-27	33.49	adjusted close
2	3	0A2I.IL	2019-11-28	33.49	adjusted close
3	4	0A2I.IL	2019-11-29	33.06	adjusted close
4	5	0A2I.IL	2019-12-02	32.61	adjusted close

df_holdings.head()



	holding_id	account_id	ticker	quantity	holding_date
0	99001	80001	FAF	16192.0	2018-10-01
1	99002	80001	CTAS	21450.0	2018-10-01
2	99003	80001	OLN	178484.0	2018-10-01
3	99004	80001	LII	16248.0	2018-10-01
4	99005	80001	PPC	228200.0	2018-10-01

✓ Quaterly Portfolio Return

```
# Ensure price_date and holding_date are in datetime format
df_pricing_daily['price_date'] = pd.to_datetime(df_pricing_daily['price_date'])
df_holdings['holding_date'] = pd.to_datetime(df_holdings['holding_date'])

# Step 1: Extract quarterly price data (last price of each quarter)
df_pricing_daily['quarter'] = df_pricing_daily['price_date'].dt.to_period('Q')
quarterly_prices = (
    df_pricing_daily.groupby(['ticker', 'quarter'])
    .agg(price_date=('price_date', 'max'), value=('value', 'last'))
    .reset_index()
)

# Step 2: Add quarter to holdings dataframe (based on holding_date)
df_holdings['quarter'] = df_holdings['holding_date'].dt.to_period('Q')

# Step 3: Merge holdings and quarterly prices on ticker and quarter
df_portfolio = pd.merge(df_holdings, quarterly_prices, on=['ticker', 'quarter'])

# Step 4: Calculate asset value per quarter
df_portfolio['asset_value'] = df_portfolio['quantity'] * df_portfolio['value']

# Step 5: Calculate total portfolio value per quarter
total_portfolio_value = (
    df_portfolio.groupby('quarter')
    .agg(portfolio_value=('asset_value', 'sum'))
    .reset_index()
)

# Step 6: Calculate portfolio weights per asset
asset_weights = (
    df_portfolio.groupby(['quarter', 'ticker'])
    .agg(asset_value=('asset_value', 'sum'))
    .reset_index()
)

# Rename 'portfolio_value' explicitly before merging if needed
total_portfolio_value = total_portfolio_value.rename(columns={'portfolio_value': 'portfolio_value_total'})

asset_weights = asset_weights.merge(
    total_portfolio_value, on='quarter', how='left'
)

asset_weights['portfolio_weight'] = (
    asset_weights['asset_value'] / asset_weights['portfolio_value_total']
)

# Step 7: Calculate quarterly returns (both continuous and discrete)
quarterly_prices['prev_value'] = (
    quarterly_prices.groupby('ticker')['value'].shift(1)
)
quarterly_prices['return_continuous'] = np.log(quarterly_prices['value'] / quarterly_prices['prev_value'])
quarterly_prices['return_discrete'] = (
    (quarterly_prices['value'] - quarterly_prices['prev_value']) / quarterly_prices['prev_value']
)

# Step 8: Merge returns with portfolio weights
returns_with_weights = pd.merge(
    quarterly_prices, asset_weights, on=['ticker', 'quarter'], how='inner'
)

# Step 9: Calculate weighted portfolio returns per quarter
weighted_returns = (
    returns_with_weights.groupby('quarter').apply(
        lambda x: pd.Series({
            'portfolio_return_continuous': np.sum(x['return_continuous'] * x['portfolio_weight']),
            'portfolio_return_discrete': np.sum(x['return_discrete'] * x['portfolio_weight'])
        })
    )
)
```

```
    })
    ).reset_index()
)

# Step 10: Display results
pd.set_option('display.float_format', '{:.6f}'.format)
display(weighted_returns)
```

```
➡ /usr/local/lib/python3.10/dist-packages/pandas/core/arraylike.py:399: RuntimeWarning: divide by zero encountered
    result = getattr(ufunc, method)(*inputs, **kwargs)
/usr/local/lib/python3.10/dist-packages/pandas/core/arraylike.py:399: RuntimeWarning: invalid value encountered i
    result = getattr(ufunc, method)(*inputs, **kwargs)
<ipython-input-21-a68172c6e5eb>:63: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns.
    returns_with_weights.groupby('quarter').apply(
```

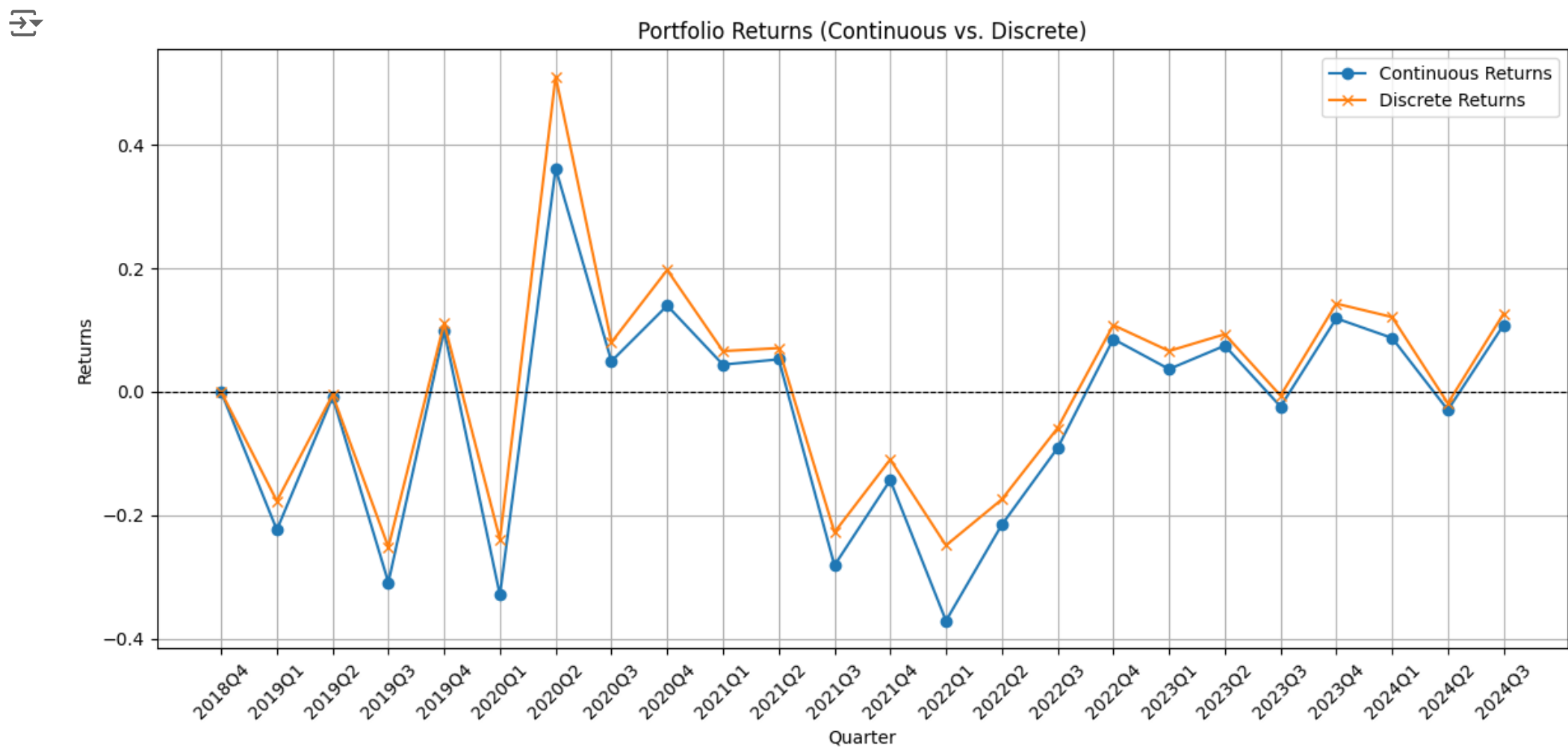
	quarter	portfolio_return_continuous	portfolio_return_discrete
0	2018Q4	0.000000	0.000000
1	2019Q1	-0.223699	-0.176917
2	2019Q2	-0.008153	-0.005041
3	2019Q3	-0.308930	-0.251772
4	2019Q4	0.099170	0.111204
5	2020Q1	-0.329135	-0.239146
6	2020Q2	0.362112	0.510824
7	2020Q3	0.049567	0.080123
8	2020Q4	0.139729	0.197425
9	2021Q1	0.044020	0.065988
10	2021Q2	0.052604	0.070810
11	2021Q3	-0.281347	-0.227696
12	2021Q4	-0.143153	-0.109708
13	2022Q1	-0.371059	-0.248918
14	2022Q2	-0.215644	-0.174466
15	2022Q3	-0.091323	-0.058975
16	2022Q4	0.085533	0.108082
17	2023Q1	0.036824	0.066165
18	2023Q2	0.074759	0.093324
19	2023Q3	-0.023923	-0.006201
20	2023Q4	0.118995	0.142901
21	2024Q1	0.087264	0.121161
22	2024Q2	-0.029498	-0.018601
23	2024Q3	0.108351	0.126723

- The quarterly returns of the portfolio show erratic performance, with notable dips (Q1 2021, with -14% continuous return and -11% discrete return) and periods of notable increase (Q2 2020, with 36% continuous return and 51% discrete return). With notable gains during recovery periods and losses during downturns, the data points to sensitivity to market conditions. A more balanced portfolio performance is indicated by recent quarters (such as Q3 2024) with moderate and steady growth (10% continuous and 13% discrete). This emphasizes how important it is to strategically rebalance during volatile times and take advantage of growth possibilities during recovery periods in order to maintain returns.

▼ Graph for Quaterly Portfolio Return

```
# Ensure the 'quarter' column is converted to string for plotting
weighted_returns['quarter'] = weighted_returns['quarter'].astype(str)

# Plot Portfolio Continuous and Discrete Returns
plt.figure(figsize=(12, 6))
plt.plot(weighted_returns['quarter'], weighted_returns['portfolio_return_continuous'], marker='o', label='Continuous')
plt.plot(weighted_returns['quarter'], weighted_returns['portfolio_return_discrete'], marker='x', label='Discrete Returns')
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.title('Portfolio Returns (Continuous vs. Discrete)')
plt.xlabel('Quarter')
plt.ylabel('Returns')
plt.xticks(rotation=45)
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```



- With notable peaks and troughs throughout time, the plotted quarterly portfolio returns show notable volatility. Sharp drops, like Q1 2021, show the portfolio's vulnerability to wider market downturns, while notable rises, like Q2 2020, show strong gains during market recoveries. The tight alignment of continuous and discrete returns shows that logarithmic and percentage-based computations are consistent. A more stable performance of the portfolio is indicated by the last quarters' low growth and decreased volatility. These patterns point to ways to take advantage of times of rapid expansion while reducing losses during recessions by using better risk management and diversification techniques.